

基于 K1MusePiPro 的复杂受限空间离线 认知边缘智能终端

摘要

本作品面向地下管廊、仓储窄巷、设备间等 GPS 拒止、通信受限、低照度及遮挡干扰场景，构建了一套基于 **SpacemiT K1** 的边缘智能巡检与自主排障辅助机器人系统。系统以履带式移动底盘为载体，通过 N10P 激光雷达、D435 RGB-D 相机和底盘里程计形成多源感知输入，在安全速度约束下完成低速 **SLAM-Frontier** 自主探测与环境信息采集。K1 边缘端部署 **YOLOv8n** 风险识别模型，对 crack、corrosion、leakage、blockage 等典型风险进行实时检测，并融合目标检测框、D435 深度、相机内参以及 odom/map 位姿，将二维图像目标转换为具有类别、置信度和空间坐标的风险地图点。系统通过同类近邻合并与时空去重，减少连续视频检测造成的风险点重复记录，最后结合本地部署的大语言模型进行风险报告的生成。

在规划执行层，系统将底盘自主探测与机械臂处置分开组织：未知环境从起点出发后，SLAM 根据 N10P 雷达和里程计增量生成 occupancy grid；探索模块提取已知空闲区与未知区边界处的 frontier，对 frontier 聚类并按照信息增益、距离和安全代价打分，选择最优探索目标；随后由 **RRT/A*/Nav2** 在当前地图中生成可行路径，并交给底盘安全守护执行。风险地图点进入机械臂侧后，**MoveIt+RL** 用于机械臂规划和高层动作原语训练，使系统形成“**SLAM-Frontier 自主探测—RRT/A*/Nav2 路径生成—风险识别—空间落图—MoveIt+RL 机械臂规划—报告生成—结果可视化**”的闭环。

关键词：进迭时空 K1 MUSE Pi Pro；边缘 AI；YOLOv8n；SLAM-Frontier 自主探测；RRT/A*/Nav2 路径生成；MoveIt+RL；本地大语言模型；风险识别

第一部分 作品概述

1.1 功能与特性

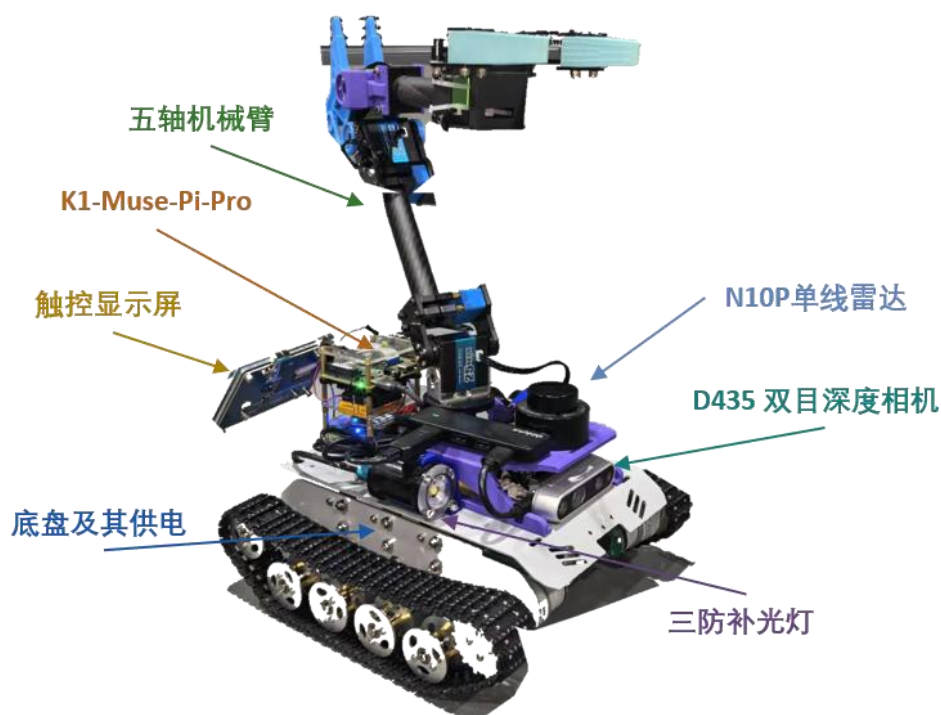


图 1-1 作品概览图

本作品构建了一套基于 K1 Muse Pi Pro 的复杂受限空间离线认知边缘智能终端，具备环境建图、风险识别、空间定位、处置规划和报告生成等功能。系统通过履带底盘搭载 N10P 激光雷达、D435 RGB-D 相机、里程计、补光单元和总线舵机机械臂，在 GPS 拒止、弱网或断网环境下完成低速安全移动与二维地图构建。D435 采集彩色图像、深度信息和相机内参，K1 端运行 YOLOv8n 模型，识别裂缝、锈蚀、渗漏和障碍物，并融合检测框、深度及 odom/map 位姿生成风险地图点。

作品具有端侧离线运行、多源感知融合、风险空间化、安全受控执行和模块化可视化等特性。各模块可独立启动和恢复，底盘运动及机械臂动作均受安全门控约束；识别、落图、规划、报告和展示均在本地完成，不依赖云端 API，可适应通信不稳定、照度变化和狭窄遮挡等复杂现场环境。

1.2 应用领域

作品主要面向地下管廊、电力沟道、仓储窄巷、设备间、工业厂房边角和应急侦察等复杂受限空间。这类环境通常存在 GPS 信号不可用、无线通信不稳定、照度不足、遮挡严重、通道狭窄和障碍密集等问题，人工巡检不仅作业效率较低，还容易出现风险漏检、位置记录不准确和处置反馈不及时等情况。系统可利用履带底盘进入人员不便长期停留的区域，通过激光雷达完成环境建图，通过 RGB-D 视觉识别裂缝、锈蚀、渗漏和通道障碍等应急情况，并将风险信息标注到二维地图中，为后续人工检修或机械臂辅助处置提供依据。由于风险识别、坐标计算、轨迹规划和报告生成均可在 K1 本地完成，因此系统在弱网、断网和云端服务不可用时仍能够保持基本巡检能力，也可进一步拓展到灾后搜索、危险区域先期探测和设备舱状态检查等场景。

1.3 主要技术特点

作品采用“多源感知—边缘认知—空间落图—规划执行—报告反馈”的分层闭环架构，如下所示：

1. **感知层**融合 N10P 激光雷达、D435 RGB-D 相机和底盘里程计，实现二维建图、环境测距和视觉数据采集。
2. **认知层**在 K1 本地部署 YOLOv8n ONNX 模型，利用置信度与深度双门限筛选有效风险，并结合几何关系和 odom/map 完成风险点空间化。
3. **规划层**分为两个边界清晰的部分：RRT/A*/Nav2 面向建图阶段的探索路径与采样目标生成，MoveIt+RL 面向机械臂动作空间目标生成。风险地图点进入机械臂模块后，由 MoveIt 完成运动学求解、限位与碰撞检测，由 RL/动作原语决定观察、拍摄、处置预备、复核及安全返回等高层候选动作。
4. **报告层**使用本地 LLM 或确定性基线生成结构化风险处置清单。

系统还采用单相机、单推理进程、独立运行目录和统一日志归档机制，降低相机、推理单元和界面显示之间的资源竞争，提高连续运行和现场演示的稳定性。

1.4 主要性能指标

表 1-1 主要性能指标示意表

指标	结果	说明
视觉模型	YOLOv8n, 3,006,628 参数	轻量目标检测模型
检测精度	mAP50=0.949, mAP50-95=0.673	D435 风险数据集验证
模型类别	crack/corrosion/leakage/blockage	覆盖多种不同类型风险
K1 推理	约 9-16 FPS 可视化展示	SpaceMIT EP 实机演示
LLM 速度	输出速度约 5.8 tokens/s	K1 本地 CLI 录屏
资源管理	单 D435/单 EP 推理进程	避免相机、EP 资源竞争
风险地图	6 个最终处置点	同类近邻合并后输出

1.5 主要创新点

系统融合 YOLOv8n、D435 深度图及底盘位姿，将 2D 风险目标转化为具空间坐标、置信度的 3D 风险点，经时空去重后，实现风险“定位与优先级排序”的空间化表达。

风险地图作为感知与执行的统一数据接口，将风险信息转化为机械臂任务目标。RRT/A*/Nav2 主要服务于建图阶段的探索路径和底盘候选运动；机械臂侧由 MoveIt 框架与 RL 动作原语完成运动学求解、限位与碰撞检测，生成观察、拍摄、处置、复核及安全返回等候选轨迹，建立机械臂动作与风险位置、类别的映射关系。

强化学习与动作原语训练约束在高层语义动作空间内，策略输出可解释动作用于控制，且必须通过安全门控校验。该设计解耦了学习策略与底层硬件执行，避免黑箱策略绕过安全控制，保障系统边界安全。

全流程在 RISC-V 架构的 K1 Muse Pi Pro 本地闭环运行，涵盖数据采集、风险识别与空间化、地图维护、轨迹生成、报告生成及 Dashboard 可视化。系统不依赖云端 API，具备弱网/断网持续运行、低时延、数据不出现场及模块独立恢复等边缘端工程优势。

1.6 设计流程



图 1-2 系统设计流程

本系统围绕“SLAM-Frontier 自主探测与 RRT/A*/Nav2 路径生成—YOLOv8n 本地风险识别—风险空间化—MoveIt+RL 机械臂处置候选—本地 LLM 报告”的链路展开设计。

第二部分 系统组成及功能说明

2.1 整体介绍

如图 2-1 所示，系统由硬件层、ROS2 中间层、AI 推理层、规划执行层和可视化报告层组成。硬件层提供移动、感知和执行能力；ROS2 中间层组织 /scan、/odom、D435 图像和风险事件；AI 推理层完成 YOLO 风险识别和本地 LLM 生成；规划执行层中，RRT/A*/Nav2 对应建图阶段的探索路径与底盘候选运动，MoveIt+RL 对应机械臂轨迹候选和动作原语；可视化层输出 dashboard、风险地图和报告。

系统依次完成多源感知、建图、风险识别、空间落图、机械臂规划、本地报告和可视化输出。建图阶段由 RRT/A*/Nav2 生成探索路径或候选运动，并通过安全守护节点约束底盘速度；风险点经过去重后送入 MoveIt+RL 机械臂规划模块，生成接近、观察、处置预备和回安全位轨迹，并通过多级安全门控约束执行。最终，本地 LLM 生成包含类别、坐标、置信度和处置建议的结构化报告，并在 K1 Dashboard 及 Windows 浏览器中展示全过程。

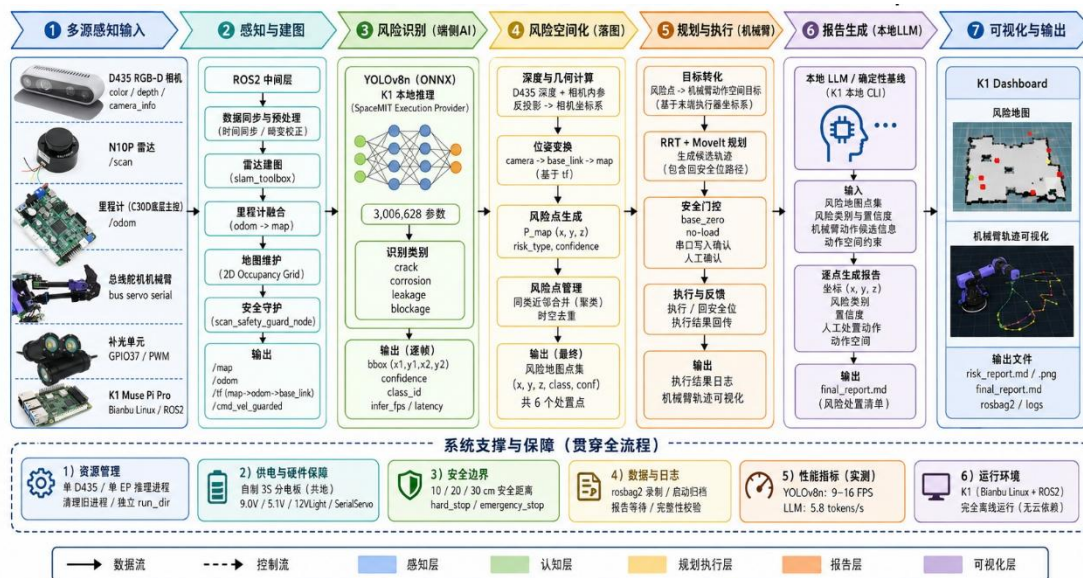


图 2-1 多源感知到风险处置的流程图

2.2 硬件系统介绍

2.2.1 硬件整体介绍；

本系统硬件部分由机械结构、供电系统、感知系统、运动执行系统和边缘计算系统共同组成，如图 2-2 所示。各模块围绕 K1 Muse Pi Pro 构成完整的移动巡检与风险处置平台。机械部分主要包括履带式底盘、左右履带组件、五轴机械臂、各级传动轴、关节连接件、传感器支架以及机械限位件。履带底盘承担整车移动与设备承载任务，可通过左右履带差速实现直行、转向和原地旋转；机械臂安装于底盘上方，通过基座、肩部、肘部、腕部和末端夹爪等关节完成风险点接近、观察、夹取、推移及回安全位等动作。各级连接件和安装限位件用于保证结构刚度、约束关节活动范围，并避免机械臂运动过程中与相机、雷达、底盘及线束发生干涉。 [1][2]

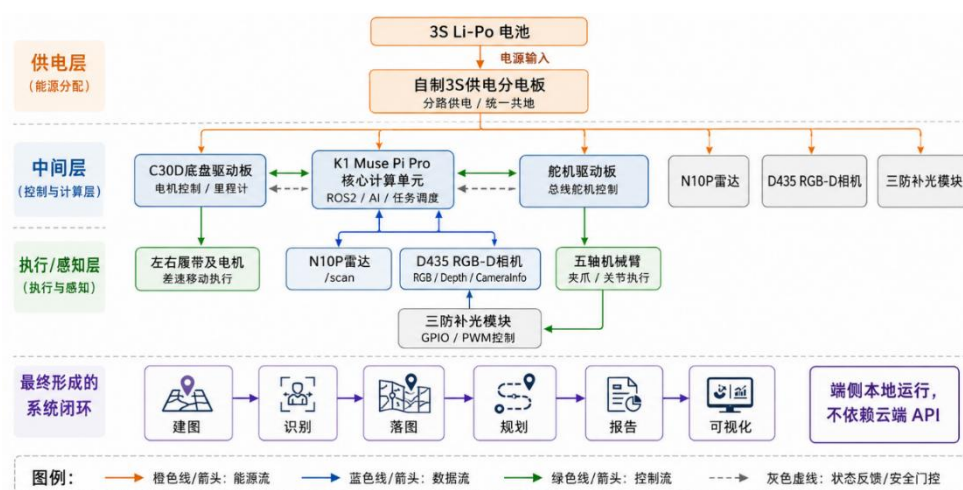


图 2-2 系统硬件组成及模块连接关系框图

电路系统主要由大容量 3S Li-Po 电池、自制 3S 供电分电板、C30D ROS 小车驱动板、舵机驱动板、N10P 单线激光雷达、Intel RealSense D435 RGB-D 深度相机、三防补光模块、总线舵机机械臂以及 K1 Muse Pi Pro 国产 RISC-V 开发板组成。3S Li-Po 电池作为整车能源输入，经自制分电板完成分路供电和统一共地，分别向底盘驱动板、K1 计算单元、D435 相机、N10P 雷达、补光灯和机械臂供电。分电板将不同电压等级与负载支路集中管理，降低临时飞线、接触不良和参考地不一致引起的复位、通信中断及图像采集异常。

K1 Muse Pi Pro 是系统的核心计算与任务调度单元，运行 Bianbu Linux 和

ROS2，负责接收 N10P 雷达、D435 相机和底盘里程计数据，并完成二维建图、风险识别、空间坐标转换、机械臂规划、本地报告生成及界面可视化。N10P 雷达向 K1 提供二维激光扫描数据，用于 SLAM 建图、障碍距离判断和底盘安全守护；D435 相机输出 RGB 图像、深度图和相机内参，用于 YOLO 风险识别、目标距离估计和风险点三维反投影；C30D ROS 驱动板读取左右电机编码器并控制履带电机，同时向 ROS2 发布底盘里程计信息，为建图和风险点坐标转换提供车辆位姿。

在控制关系上，K1 输出的底盘速度指令首先进入安全守护节点，经障碍距离、速度上限和急停条件判断后，再由 C30D 驱动板控制左右履带运动。机械臂控制指令则由 K1 发送至舵机驱动板，经总线通信控制各关节舵机；实际执行前还需通过底盘静止、机械臂工作空间、关节限位、无负载测试和人工确认等安全条件。补光模块安装在相机附近，由 K1 通过 GPIO 或 PWM 控制，在低照度环境中为 D435 的彩色图像采集和风险识别提供稳定照明。

系统形成了“电池与分电板提供能源—传感器采集环境信息—K1 完成数据融合与决策—底盘和机械臂执行动作—传感器反馈运行结果”的硬件闭环。各子模块既可独立调试，又通过 ROS2 话题、串口、USB 和电源总线协同工作，使整车具备移动、感知、计算、规划、执行和反馈的一体化能力。

2.2.2 机械设计介绍

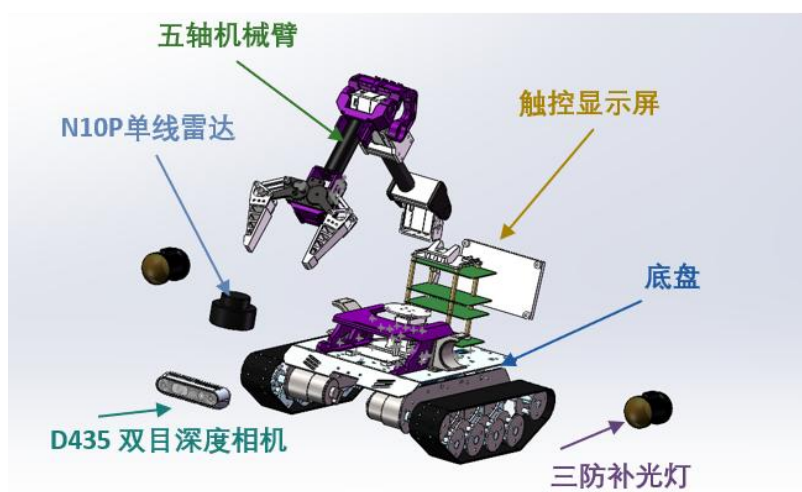


图 2-3 整车设计图

如图 2-3 所示，机械系统由履带式差速底盘、左右履带组件、驱动轮与承重轮、五轴机械臂、末端夹爪、各关节传动与连接组件、传感器安装支架以及机械限位件组成。履带底盘作为整机移动载体，可通过左右履带差速实现直行、转向和原地旋转，相较普通轮式底盘具有较大的接地面积，适合低速通过狭窄通道、轻微起伏地面和存在碎小障碍物的巡检区域。



图 2-4 夹爪以及机械臂具体样貌图

如图 2-4，五轴机械臂安装于底盘上方，由基座回转、肩部、肘部、腕部和末端夹爪等关节构成，可完成接近风险点、调整末端姿态、夹持或推移轻型障碍物以及返回安全位等动作。各级连接件承担机械臂载荷传递和关节定位作用，限位件用于约束关节运动范围，防止机械臂与底盘、相机、雷达及线束发生干涉。机械臂规划过程中还设置工作空间、关节软限位、回安全位和底盘静止等约束，以提高实际运行安全性。D435 布置在前向视角，N10P 雷达提供水平扫描，机械臂安装在车体上方并覆盖前方近距离处置区域。机械臂默认回到 home 安全位，MoveIt+RL 规划出的动作包含安全位路径，避免机械臂长时间停留在伸展姿态。

2.2.3 电路各模块介绍

电气连接围绕 K1 主控和自制 3S 机器人供电分电板展开。D435 通过 USB 提供 RGB-D 数据，N10P 雷达通过驱动工作区发布 LaserScan，底盘控制器通过安全速度链路接收速度命令，机械臂总线控制器通过串口发送舵机指令，补光单元通过 GPIO/PWM 控制。演示中避免同时启动多个 D435/EP 推理进程，防止相机和执行单元资源竞争。

2.2.3.1 自制 3S ROBOT PDB 供电分电板

如图 2-5，可见自制供电板以 3S LiPo 9.0-12.6 V 为输入，采用所有 GND 共地的车载供电拓扑，在同一块 PCB 上整理底盘、电机、K1、RealSense、补光、机械臂总线舵机和外设接口。供电板正面标注 9.0VOut、5.1VOut、12VLight、SerialServo、BAT IN 等端口，便于装车时按功能分区接线。

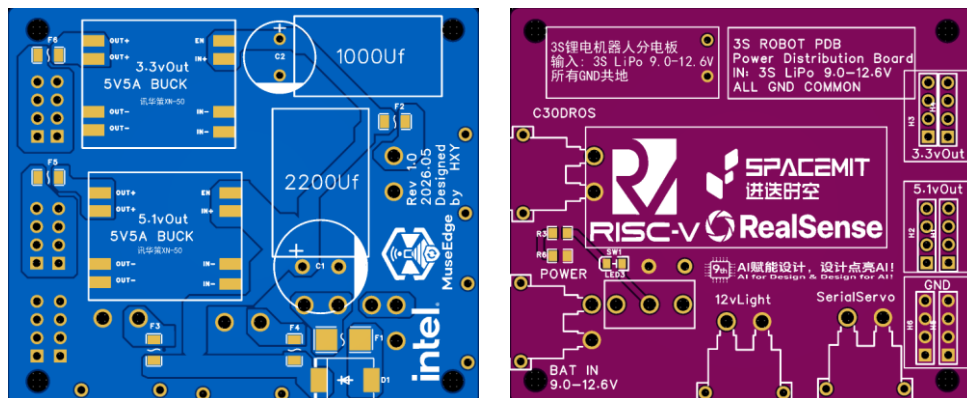


图 2-5 供电板电路 2D 图

该供电板的工程价值在于把原本分散的电源线、降压模块、共地线和外设接口收敛为可维护的电气节点（如表 2-1）。对于履带底盘和机械臂组合系统，电机启动、D435 采集、K1 推理、补光和舵机响应会同时产生电流需求；集中分电可以减少临时飞线、接触不良和地线不一致带来的复位或噪声问题。

表 2-1 自制 3S ROBOT PDB 供电分电板接口与电气功能

接口区域	电气功能	接入对象	设计作用
BAT IN	3S LiPo 输入	主电池/电源总线	为整车提供统一输入电源
9.0VOut	中压输出	车载外设/执行支路	为需要较高裕量的模块供电
5.1VOut	低压逻辑输出	显示/传感/辅助逻辑	为低压外设提供稳定供电
12VLight	补光输出	管腔补光灯	在低照度场景中提供补光
SerialServo	舵机总线接口	机械臂总线舵机	为机械臂提供供电与接口整理
GND	所有地线共地	K1、传感器、执行器	减少参考地不一致导致异常

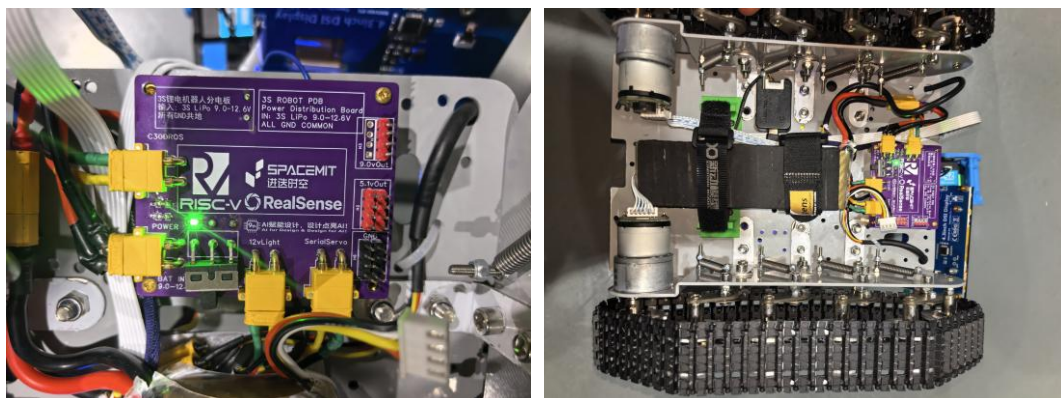


图 2-6 自制 3S ROBOT PDB 供电分电板局部实物图及其安装位

如图 2-6，板上 9.0VOut 可为车载外设和部分执行支路提供稳定输出，5.1VOut 面向低压逻辑外设，12VLight 面向补光负载，SerialServo 面向机械臂舵机总线。各路输出在物理位置上分开布线，现场检修时能够快速判断某一路负载是否接入、是否松动、是否需要单独断开。

供电板还承担演示可靠性保障作用。K1 端需要同时运行 ROS2、RealSense、YOLOEP、dashboard 和 LLM CLI，若供电和共地不稳定，最容易表现为 D435 丢失、推理进程异常、串口通信不稳定或底盘响应间歇。自制分电板把这些问题前置到硬件层处理，提高了连续录屏和多窗口展示的稳定性的，其中图 2-6 展示了实物板图以及其具体在车上安装的位置。

2.2.3.2. K1 Muse Pi Pro 边缘计算单元

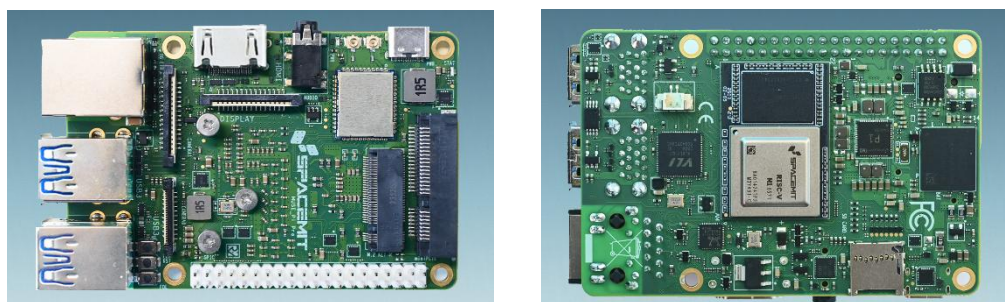


图 2-7 K1 Muse Pi Pro 主控板

K1 Muse Pi Pro 搭载 SpacemiT K1 处理器，采用 64 位八核 X60 RISC-V CPU，支持 RV64GCVB 指令集组合、RVA22 应用处理器规范及 RVV 1.0 向量扩展，并集成约 2.0 TOPS 的 AI 计算能力；K1 平台最高可支持 16 GB LPDDR4 内存。

其中，RV64 表示处理器采用 64 位 RISC-V 基础架构；G 代表面向通用计算

的基础整数、乘除法、原子操作以及单、双精度浮点运算扩展，可满足 Linux、ROS2 和复杂应用程序运行需求；C 为压缩指令扩展，可通过更紧凑的指令编码减小程序体积和指令访存压力；V 为 RVV 1.0 向量扩展，可使用单条向量指令并行处理多组数据，适合图像预处理、矩阵运算、卷积计算和传感器数据处理；B 为位操作扩展，可提升掩码、移位、计数及部分整数数据处理效率。RVA22 则为 64 位应用处理器提供相对统一的指令与软件运行基线，有利于操作系统、编译器及应用软件在不同 RISC-V 平台之间适配。

在 AI 推理方面，系统将训练完成的 YOLOv8n 模型导出为 ONNX 格式，并通过 ONNX Runtime 接入 SpaceMIT Execution Provider。需要注意，Execution Provider (EP) 并不是一颗独立的硬件“核心”，而是 ONNX Runtime 与 K1 底层硬件加速库之间的软件执行后端。模型加载后，EP 会分析计算图，将自身支持的算子或子图分配给 K1 的加速计算资源执行；不能由 EP 处理的节点则可回退至 CPU 执行。通过这种计算图分配机制，应用层仍使用统一的 ONNX Runtime 接口，同时能够利用 K1 平台的专用加速能力，降低纯 CPU 推理负担并改善模型推理延迟。ONNX Runtime 官方将 EP 定义为面向 CPU、GPU、FPGA 或专用 NPU 等硬件的可扩展执行接口，并支持按优先级组合多个执行后端。

在本系统中，K1 运行 Bianbu Linux 与 ROS2，承担 D435 RGB-D 数据读取、N10P 雷达与里程计数据接入、YOLOv8n 端侧推理、风险点空间化、二维地图和日志管理、SLAM-Frontier 自主探测控制、MoveIt+RL 机械臂规划接口、本地 LLM 报告生成以及 Dashboard 可视化等任务，是整车的数据汇聚、边缘计算和任务调度中心。系统采用“SpaceMIT EP 优先、CPU 回退”的推理方式，并限制同一时刻仅运行一组 D435 采集与 EP 推理进程，以减少 USB 带宽、内存、推理后端和显示模块之间的资源竞争，保证连续识别和现场演示的稳定性。

2.2.3.3. Intel RealSense D435 深度相机



图 2-8 D435 双目深度相机

D435 采用主动双目立体深度技术，深度传感器使用全局快门，适合机器人运动过程中的深度采集。其深度视场角约为 $87^{\circ} \times 58^{\circ}$ ，推荐工作距离为 0.3~3 m，最大分辨率下最小测距距离约为 0.28 m，2 m 处标称深度误差小于 2%；深度输出最高可达 1280×720、90 fps，RGB 图像最高为 1920×1080、30 fps。相机尺寸约 90 mm×25 mm×25 mm，通过 USB-C 3.1 Gen 1 连接。其最大探测范围可达到约 10 m，但实际有效距离会受到目标材质、环境光照和红外反射条件影响。

本项目实际采用较低的 RGB 与深度分辨率，以降低 K1 端 USB 带宽、图像预处理和 YOLO 推理负担。D435 输出彩色图像、对齐深度图和相机内参，分别用于风险分类、目标距离估计和三维坐标反投影。

2.2.2.4. N10P 单线激光雷达



图 2-9 N10P 激光雷达

系统采用 N10P 单线二维激光雷达作为环境建图与底盘安全感知单元。该雷达基于 TOF 飞行时间测距原理，使用 905 nm 激光光源，可实现 360° 水平扫描，并输出距离、角度和回波光强信息。N10P 的输出距离分辨率为 1 mm，最小测距距离为 0.1 m；对于反射率约 70% 的目标，测距精度约为 ± 3 cm。其测量半径在白色目标条件下可达 25 m，在黑色目标条件下约为 11 m，能够满足小型管廊、仓储通道和设备间二维建图需求。

雷达测量频率为 5400 次/s，角分辨率可在 $0.4^{\circ} \sim 0.8^{\circ}$ 范围内调节，扫描频率为 6~12 Hz 可调。设备采用 5 V 直流供电，典型工作电流约 360 mA，支持网口和标准串口通信，串口波特率为 460800 bps。N10P 支持 ROS1、ROS2、Windows 上位机以及 Python 数据采集，可通过 ROS2 发布 /scan 激光扫描话题，为 slam_toolbox 二维建图、前向障碍检测和安全速度裁剪提供实时数据。

2.2.2.5 C30D ROS 底盘驱动板与里程计

C30D 底层控制板负责接收 ROS 端速度指令，读取左右履带电机编码器数据，并执行左右电机闭环调速。主控板根据目标线速度和角速度完成差速运动解算，再通过电机驱动接口控制两侧履带，实现直行、转向和原地旋转。编码器累计值进一步转换为底盘里程计，并通过 ROS 发布/odom，为建图和风险点坐标转换提供车辆位姿信息。

控制板主控所结合的履带形式车辆底盘采用采用 MG513 永磁有刷直流减速电机作为左右履带驱动单元。电机额定电压为 12 V，额定功率约为 4 W，额定电流约为 360 mA，堵转电流约为 3.2 A。电机减速比为 1：30，减速前转速约为 11000 r/min，减速后输出转速为 366 ± 26 r/min。其额定输出转矩约为 1 kg·cm，堵转转矩约为 4.5 kg·cm，可为履带底盘低速行驶、原地转向及小范围建图提供驱动力。

电机尾部集成霍尔编码器，编码器线数为 13 PPR，供电电压为 3.3~5 V，采用 XH2.54 接口，并具备上拉整形功能。C30D ROS 底层主控读取左右电机编码器脉冲，计算两侧履带转速和累计位移，再依据差速运动学模型生成底盘里程计 /odom。该里程计进一步参与二维建图、车辆位姿估计以及风险点由相机坐标系向 odom/map 坐标系的转换。

2.2.2.6 五轴总线舵机机械臂

机械臂采用总线舵机串联控制方式，各关节舵机通过独立 ID 共享供电与通信总线，由 K1 Muse Pi Pro 经舵机驱动板统一下发目标位置、运动时间和动作序列。相比传统逐路 PWM 控制，总线控制能够减少线束数量，便于多关节同步运动、动作编排和状态管理。

如图 2-4，机械臂肩部、肘部、腕部及末端夹爪等执行单元组成。关节驱动采用 HTD-35H 总线舵机，其工作电压为 9~12.6 V，推荐在 11.1 V 电压下运行。单个舵机重量约 64 g，外形尺寸为 51.1 mm×20.14 mm×40 mm，转动范围为 0°~240°；在 11.1 V 条件下，空载运动速度约为 0.18 s/60°，堵转扭矩可达 35 kg·cm，可满足机械臂关节转动、姿态保持及轻型目标操作需求。

机械臂末端可配置二指、三指或四指电动夹爪。夹爪整体尺寸约为 63

mm×63 mm×165.5 mm，夹持开口范围为 10~120 mm，供电电压为 DC 12~24 V，最大工作频率低于 40 次/min。不同夹爪形式的重量和标称抓取能力有所不同：二指夹爪重量约 370 g，标称抓取重量约 0.6 kg；三指夹爪重量约 420 g，标称抓取重量约 1 kg；四指或十字四指夹爪重量约 510 g，标称抓取重量约 1.5 kg。

在系统运行过程中，风险地图点首先被转换为机械臂末端目标位姿，MoveIt+RL 根据关节范围、碰撞边界和工作空间生成候选轨迹。真实动作执行前，还需依次通过底盘静止、关节软限位、目标可达性、无负载测试、串口写入确认及人工确认等安全门控。若规划失败、目标超出工作空间或通信状态异常，系统将停止动作并返回安全状态，避免机械臂与底盘、传感器及周围环境发生碰撞。

2.2.2.7. 三防补光模块

三防补光模块安装在 D435 RGB-D 相机附近，主要用于改善地下管廊、设备间、仓储窄巷及遮挡区域中的低照度成像条件。补光灯为系统提供稳定的可见光照明，可增强 RGB 图像中的纹理、边缘和颜色信息，提高 YOLOv8n 对裂缝、锈蚀、渗漏和障碍物等目标的识别稳定性。需要说明的是，补光模块主要改善 D435 的彩色图像质量，并不等同于直接提升所有环境下的深度测量性能。

补光灯支持 7~48 V 宽电压供电，可直接接入自制 3S 供电分电板的 12VLight 支路。模块最大亮度可达 1500 lm，色温约为 5700 K，可提供接近日光色的照明效果，能够覆盖 D435 前方较大视场，减少局部照明造成的强阴影和视野盲区。根据厂家给出的峰值电流关系：

$$I_{peak} = 15/V_{in}$$

其峰值输入功率约为 15 W；在 12 V 供电条件下，理论峰值电流约为 1.25 A，因此供电支路和导线需要保留足够的电流裕量。

模块支持 PWM 调光和直接开关两种控制方式。PWM 调光信号脉宽范围为 1100~1900 μs，K1 可通过 GPIO/PWM 输出控制亮度，以适应不同距离、墙面反射率和环境照度。当不具备 PWM 控制条件时，可将黄色信号线接入正极，使补光灯以开关模式直接点亮。控制程序可根据图像平均亮度、识别置信度或任务状态调整补光强度，避免亮度过低导致目标纹理丢失，也避免近距离墙面

反光和图像过曝。

补光模块采用约 0.5m 长的 22 AWG 三芯线缆，其中红线为电源正极、黑线为电源负极、黄线为 PWM 控制信号。安装接口采用 M3 螺纹孔，可固定于相机支架或车体前部。模块在空气中的重量约为 120 g，水中等效重量约为 50 g，标称耐压深度可达 500 m，空气中最高运行温度为 55 ℃。虽然本系统主要用于陆地受限空间巡检，但其防水、防尘和耐环境特性有助于提高设备在潮湿、积水和粉尘环境中的可靠性。

2.3 软件系统介绍

2.3.1 软件整体介绍；

本系统软件部分由机器人端实时控制软件、边缘 AI 推理软件、风险地图生成软件、机械臂规划执行软件、本地报告生成软件 and 上位机辅助调试软件共同组成。各软件模块围绕 K1 Muse Pi Pro 运行的 Bianbu Linux 和 ROS2 Humble 展开，通过 ROS2 话题、Python 进程、ONNX Runtime 推理后端、HTTP 可视化服务和本地文件归档目录构成完整的软件闭环。 [1][3]

在系统运行过程中，K1 端承担主要实时任务，包括 N10P 雷达数据接入、底盘里程计读取、SLAM-Frontier 自主探测、安全速度裁剪、D435 图像与深度数据采集、YOLOv8n 风险识别、风险点空间化、MoveIt+RL 机械臂规划、dashboard 可视化以及本地 LLM 报告生成。Windows 端主要作为操作与归档终端，用于 SSH 启动任务、键盘遥控建图、查看 K1 端口页面、拉回地图和风险事件文件，并完成视频素材整理。

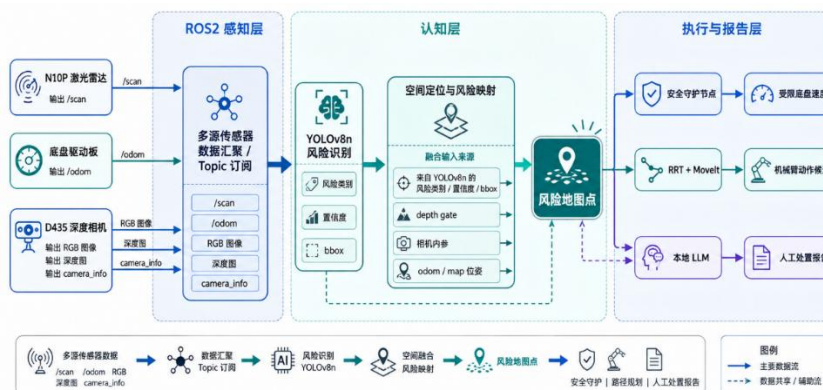


图 2-10 软件总体数据流示意图

如图 2-10，软件总体数据流为：传感器数据首先进入 ROS2 感知层，N10P 输出 `/scan`，底盘驱动板输出 `/odom`，D435 输出 RGB 图像、深度图和 `camera_info`；随后进入认知层，由 YOLOv8n 完成风险类别、置信度和 `bbox` 预测，再结合 `depth gate`、相机内参和 `odom/map` 位姿生成风险地图点；最后进入执行与报告层，由安全守护节点输出受限底盘速度，由 MoveIt+RL 生成机械臂动作候选，由本地 LLM 输出人工处置报告。

为保证现场演示稳定，软件系统采用“单相机、单推理、单运行目录”的组织方式。每一轮任务创建独立 `run_dir`，保存 `realsense.log`、`mapping_guard.log`、`yolo_ep.log`、`runtime_status.json`、`risk_events.jsonl`、风险截图、风险地图和报告文件；启动前清理旧的 D435、YOLO、HTTP 和遥控进程，避免相机、SpaceMIT EP、浏览器服务和日志文件相互竞争。

2.3.2 软件各模块介绍；

软件各模块按照“输入数据、处理逻辑、输出结果”的方式组织。顶层由 ROS2 `launch` 文件负责启动传感器、建图、安全守护和机械臂相关节点；中间层由 Python 工具负责图像推理、风险事件记录、地图点生成、运行状态写入和 `dashboard` 文件刷新；底层由 ONNX Runtime、RealSense 驱动、雷达驱动、底盘驱动板和总线舵机驱动共同提供硬件访问能力。

从功能上看，软件系统可分为六个模块：SLAM-Frontier 自主探测与安全守护模块、D435 YOLO 风险识别模块、风险空间化模块、MoveIt+RL 机械臂规划模块、本地 LLM 报告模块以及可视化与归档模块。各模块既能独立启动验证，也能在完整演示脚本中按顺序串联运行。

关键输入变量包括 `/scan`、`/odom`、`/map`、RGB 图像、`aligned depth`、`camera_info`、键盘速度指令 `/input_vel`、模型输出 `bbox/class/confidence`、机械臂当前状态和风险点列表；关键输出变量包括 `/cmd_vel_guarded`、`risk_detection.json`、`risk_point.json`、`final_risk_points.json`、`risk_map_final_rrt_rl.png`、`dashboard` 状态文件以及 `final_report.md`。

2.3.2.1 SLAM-Frontier 自主探测与安全守护模块

SLAM-Frontier 自主探测与安全守护模块负责把未知环境探索、路径生成和

收 `input_vel` -> 读取 `/scan` 前向距离 -> 判断急停/慢速/放行状态 -> 发布 `/cmd_vel_guarded` -> 同步记录 `front_min`、`odom` 和运行状态。其主要输出包括实时 `/map`、保存后的地图文件、`mapping_guard` 日志以及 `dashboard` 中显示的 `front_min` 和 `odom` 信息，图 2-13 可见对于突然出现的风险，小车也能正确识别并合理退让。

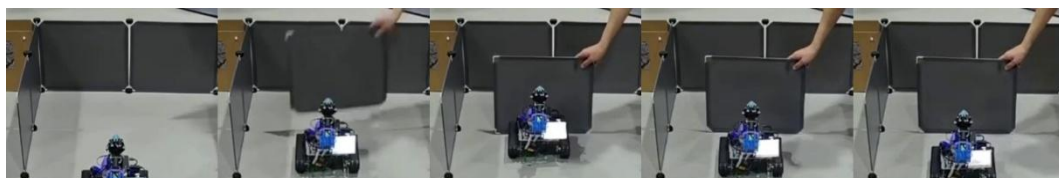


图 2-13 对于偶然出现的相关风险障碍处理（按照时序整理，相关能力同步在展示视频）

软件验证采用分阶段方式：先验证基础建图，再接入 SLAM-Frontier 自主探测与 RRT/A*/Nav2 路径生成策略，最后与风险识别、机械臂处置候选和报告生成模块联调。

完整运行时每轮任务使用独立 `run_dir`，记录 `realsense.log`、`mapping_guard.log`、`yolo_ep.log`、`runtime_status.json`、`risk_events.jsonl`、风险截图、风险地图和报告文件。该组织方式把底盘、相机、EP、HTTP 页面和报告生成的状态分开保存，便于现场出现异常时快速定位问题。

2.3.2.2 D435 YOLO 风险识别模块

D435 YOLO 风险识别模块的输入为 D435 彩色图像、对齐深度图和 `camera_info`。彩色图像经预处理后输入 YOLOv8n ONNX 模型，模型在 K1 本地通过 ONNX Runtime 调用 SpaceMITExecutionProvider，并在算子不支持时回退到 CPUExecutionProvider。模型输出包括 `bbox`、`class`、`confidence` 和推理 `latency`。 [6][7][8]

我们将真实物理环境中的风险抽象为其相对应的图片，堵塞物用相关小物块进行模拟。本模块支持 `crack`、`corrosion`、`leakage` 和 `blockage` 四类风险。实时显示时使用较低候选阈值 `--conf0.5`，保证操作者能在 `overlay` 中看到边界样本；自动报警和风险落图则采用更严格的类别门限和深度门限。

该模块的关键函数流程为：读取 D435 帧 -> 根据模型输入尺寸完成 `resize/letterbox` -> 执行 ONNX 推理 -> 后处理与 NMS -> 计算 `bbox` 中心深

度 -> 应用 confidence+depth gate -> 输出 risk_detection 事件。若事件通过门限，则保存 overlay.png、rgb.png、depth_raw.npy、depth_vis.png、camera_info.json 和 risk_detection.json。



图 2-14 YOLOv8n 模型在其测试集上的结果

模型部署后的现场校准调整了模型输出到报警/落图之间的接受边界。补光策略同样来自现场采样：默认不开动态补光，只有暗管腔 leakage 画面不稳定时才开启固定亮度补光，如图 2-14 即为 YOLOv8n 模型在测试集上结果。

现场阈值不是直接照搬训练集结果，而是通过 dry-run 采集 overlay、rgb、depth 和风险事件后人工复核得到。调试时先统计高置信样本，再查看所有识别图片，最终将 confidence、depth、类别和去重规则共同作为自动报警和落图条件。

补光策略采用人工按需开启，而不是动态调光。D435 自动曝光能够覆盖大部分亮暗变化；只有暗管腔 leakage 等低照度画面不稳定时，才开启固定亮度补光。这样可以避免动态补光改变图像分布，同时保留低照度目标的识别能力。

2.3.2.3 风险空间化模块

风险空间化模块的输入为 YOLO 输出的 bbox/class/confidence、D435 对齐深度图、camera_info 相机内参以及底盘 /odom 或 map 位姿。模块首先在 bbox

中心附近提取有效深度，得到目标在相机光学坐标系下的三维点，再结合相机到车体的近似外参和机器人当前位姿，将图像风险转换为 odom/map 坐标系中的风险点。

该模块的关键处理流程为：risk_detection -> bbox 中值深度 -> camera_info 反投影 -> base 坐标近似转换 -> odom/map 位姿叠加 -> 风险点写入。主要输出为 risk_map_point.json 、 risk_events.jsonl 、 final_risk_points.json 和 risk_map_final_rrt_rl.png。系统明确不使用人工采集距离作为最终地图坐标来源，人工距离只作为数据集覆盖信息。

为减少视频流重复检测造成的报告冗余，模块对同类风险点进行空间合并。同类风险在 0.15 m 内合并为一个最终处置点，并保留代表置信度、代表深度、代表坐标和来源事件。

风险点落图后需要与实际场景复核。最终文档引用的是复核后的风险地图和 final_risk_points.json，保证图、报告和规划输入来自同一份结构化数据。

2.3.2.4 MoveIt+RL 机械臂规划模块

MoveIt+RL 机械臂规划模块的输入为 final_risk_points.json 中的风险类别、风险坐标、深度和处置动作类型，同时读取机械臂当前状态、关节限位、工作空间边界和安全状态。该模块不接收 RRT/A*/Nav2 的底盘建图输出作为关节轨迹，而是使用复核后的风险点作为末端目标区域，由 MoveIt 完成运动学、碰撞检测和轨迹表示，由 RL/动作原语选择观察、预备、处置候选和回安全位等高层动作。
[9][11]

规划流程为：读取风险点 -> 根据类别由 RL/动作原语选择高层动作 -> 生成观察位、预备位和回安全位 -> 调用 MoveIt 完成运动学求解、碰撞检测和轨迹表示 -> 检查工作空间、关节限位、碰撞边界和底盘静止状态 -> 输出机械臂响应或处置候选。crack 对应修补/密封动作空间，corrosion 对应除锈/防腐动作空间，blockage 对应清障/移除动作空间，leakage 对应查漏/封堵动作空间。

真实执行前还需要通过 base_zero、目标可达性、串口写入确认和人工确认等安全门控。若规划失败、目标超出工作空间或通信状态异常，模块输出 failure_reason 并回到 STOP_SAFE，不向舵机发送真实接触动作。

风险类别到动作空间的映射为：**crack** 对应清理表面、裂缝修补或密封、复查边缘；**corrosion** 对应清理浮锈、打磨腐蚀层、涂覆防腐材料、复核管壁；**blockage** 对应确认障碍体积、夹取或推移、清理通道、复查通行空间；**leakage** 对应查找源头、临时封堵、擦拭干燥和二次复查。MoveIt+RL 输出的是观察位、预备位、保持拍照位和回安全位等候选轨迹。

2.3.2.5 本地 LLM 报告模块

本地 LLM 报告模块的输入为 `final_risk_points.json`、风险地图图像、风险类别动作空间和任务提示词。模块在 K1 本地通过 `llama.cpp CLI` 或确定性报告基线生成巡检结论，不调用云端 API。报告生成过程保留 `prompt` 处理速度、首字输出状态和 `generation tokens/s`，便于在视频中展示端侧大模型运行能力。 [10]

报告生成逻辑是从结构化风险点逐项生成处置建议。每个风险点需要包含编号、类别英文标签、中文含义、`odom/map` 坐标、置信度、处置动作和动作空间。例如 **crack** 对应破损修补，**corrosion** 对应除锈处理，**blockage** 对应障碍物清理，**leakage** 对应查漏和封堵。模块输出包括 `final_report.md`、`llm_stream_report_terminal.md`、`llm_stream_metrics_terminal.json` 和录屏素材。

报告提示词限定为风险探测任务：系统需要说明在具体 `odom/map` 近似位置发现了哪一类风险，以及人工应执行哪类处置动作。报告把 **crack**、**corrosion**、**blockage**、**leakage** 转换成修补、除锈、清障、查漏封堵等可执行任务。

2.3.2.6 可视化模块

可视化模块由 K1 本地 `dashboard`、Windows 浏览器页面、风险地图和 CLI 输出组成。Dashboard 通过 K1 本地 HTTP 服务发布，读取 `runtime_status.json`、YOLO overlay、雷达距离、`odom`、当前 `detections` 和风险地图文件。

如图 2-15，K1 屏幕主要用于现场展示本地闭环，Windows 端通过 8765 端口访问可同时用于远程观察、截图、拉回文件和视频整理。最终演示时，可视化模块先展示 D435 YOLO overlay 和实时帧率，再展示风险位置图，最后录制 LLM CLI 报告生成过程。这一顺序对应“看见风险、定位风险、规划处置、生成报告”的软件闭环，使系统输出从实时画面延伸到可归档的巡检结果。

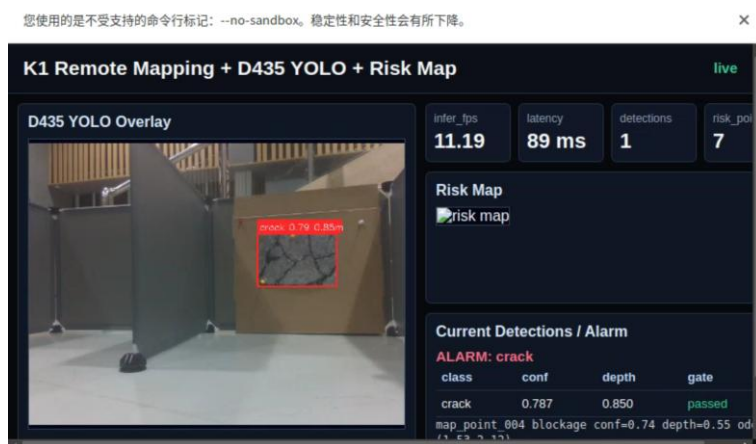


图 2-15 K1 实机演示网页

演示视频按“架构与硬件拓扑、实机 SLAM-Frontier 自主探测与 YOLO 风险识别、风险点落图、MoveIt+RL 机械臂响应、本地 LLM CLI 报告生成”的顺序组织。录屏重点保留 D435 YOLO overlay、infer_fps、latency、雷达前向距离、odom、风险地图和 LLM tokens/s，使评审能直接看到端侧推理和报告生成过程。

第三部分 完成情况及性能参数

表 3-1 实现真机的性能参数

模块	量化结果	说明
YOLO 训练	mAP50=0.949	D435 风险打印图数据集
模型规模	3,006,628 参数	YOLOv8n 轻量模型
K1 推理	约 9-16 FPS	SpaceMIT EP 演示
LLM 生成	约 5.8 tokens/s	K1 本地 CLI
地图	0.05 m/pixel	map_saver_cli 输出
风险点	6 个处置点	同类近邻合并
安全边界	10/20/30 cm	停止/慢速/受控上限

3.1 整体介绍



图 3-1 系统实物正面照片 及其 斜 45°全局照片

最终系统已经形成以 K1 MUSE Pi Pro 为边缘计算核心的移动风险探测机器人实物。整机由履带式 Tank 底盘、前向 D435 RGB-D 相机、N10P 二维雷

达、总线舵机机械臂、自制 3S 机器人供电分电板、补光单元和 K1 本地显示端组成，各模块围绕“现场感知—本地推理—地图定位—处置建议—报告生成”的闭环展开，其真机参数如表 3-1 所示。

从系统正面观察，D435 与雷达位于主要感知方向，用于获取前方图像、深度和障碍距离；从斜 45°全局观察，可以看到底盘、供电、线束、相机、雷达和上层计算单元之间的空间关系。履带底盘提供低速稳定移动能力，K1 负责运行 ROS2、YOLO、风险落图、dashboard 和本地 LLM，Windows 端用于遥控、日志观察和结果拉回。

演示流程中，系统先完成 SLAM-Frontier 辅助探测并保存地图，同时 D435 YOLO 风险识别，将 crack、corrosion、blockage、leakage 等风险转换为地图坐标点，最后在 K1 本地生成基于本地 LLM 生成的风险处置报告。该实现重点体现了端侧 AI 推理、实时可视化、局部安全闭环和任务结果归档能力。

3.2 工程成果

本系统围绕“SLAM-Frontier 自主探测与 RRT/A*/Nav2 路径生成—YOLOv8n 本地风险识别—风险空间化—MoveIt+RL 机械臂处置候选—本地 LLM 报告”的链路展开设计。

3.2.1 机械成果



图 3-2 履带底盘、D435、雷达与机械臂整体机械结构

机械成果包括履带式 Tank 底盘、前向 D435 安装位、二维雷达安装位、总线舵机机械臂、线束固定结构和补光支撑结构。履带底盘能够完成前进、后退和原地转向，适合在桌面级管廊/狭窄通道样机环境中进行低速巡检演示。

D435 采用前向固定安装，保证 RGB 图像、深度图和风险目标处于同一观察方向；二维雷达用于建图和安全守护，配合 /scan 前向距离判断底盘是否应急停、慢速或放行。机械臂部分以 MoveIt+RL 规划和安全响应为主，动作结束后回到安全位，当前展示重点是处置动作空间和安全响应链路。

机械结构的实际收益在于：感知方向明确、底盘运动稳定、机械臂与底盘之间保留安全间距，现场排障时可以先由人工确认风险点，再决定是否触发机械臂响应或后续真实清障实验。

3.2.2 电路成果；

电路成果以自制 3S 机器人供电分电板为核心。该分电板支持 3S LiPo 9.0-12.6V 输入，所有 GND 共地，并提供 9.0VOut、5.1VOut、12VLight、SerialServo 等接口，把底盘、K1、D435、补光和机械臂舵机供电集中到一个可检查的硬件节点。

相比临时接线，分电板降低了演示时排查供电问题的难度。现场可以根据接口丝印和状态指示快速确认电池输入、外设供电、补光输出和舵机总线是否处于预期状态，也便于后续将补光、机械臂和传感器分路管理。

电气连接遵循“主控稳定优先、执行器分路供电、信号共地”的原则。K1 与 D435 主要负责感知和推理，底盘控制链路负责运动执行，机械臂总线负责处置动作，补光单元作为可开关常量光源，在暗环境风险识别时可独立开启。

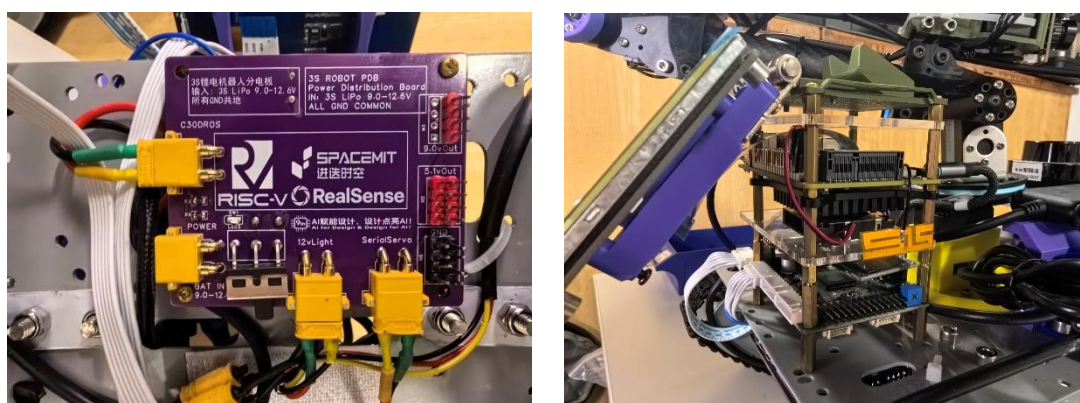


图 3-3 自制 3S 机器人供电分电板实物照片 及其主控板层次堆叠效果

3.2.3 软件成果

软件成果由 ROS2 建图与安全守护、D435 YOLO 风险识别、风险点空间化、

dashboard 可视化、本地 LLM 报告和演示收尾脚本组成。系统运行时，/input_vel 先进入安全守护节点，结合 /scan 前向距离得到 /cmd_vel_guarded；D435 图像与深度进入 YOLO 风险识别节点；风险事件写入同一 run_dir，供 dashboard、风险地图和报告模块共同读取。

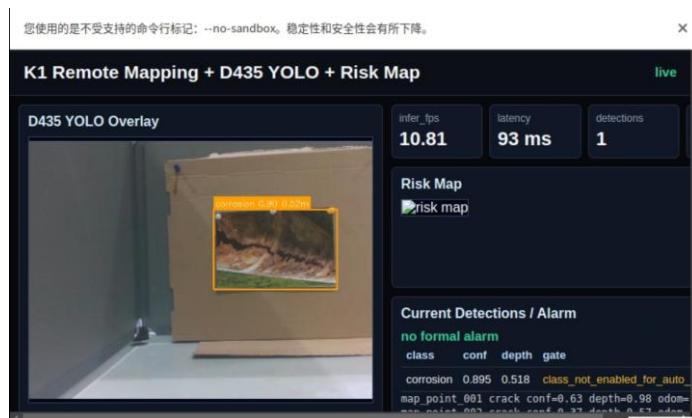


图 3-4 K1 实机 dashboard: YOLO overlay、帧率、雷达距离和报警信息

dashboard 是最终展示的主要软件界面（图 3-4），能够显示 D435 YOLO overlay、infer_fps、latency、front_min、odom、当前 detections/alarm 和 risk map。该界面既可在 K1 本地浏览器打开，也可在 Windows 端通过 K1 的 8765 端口访问，保证现场展示和远程观察看到同一份数据。风险排除结束后，收尾脚本保存地图、风险帧、风险点 JSON、风险位置图（图 3-5）、LLM 报告（图 3-6）和运行指标。这样每次实机演示都能形成可回放、可复核的材料，既满足比赛视频展示，也方便在报告中引用具体证据。

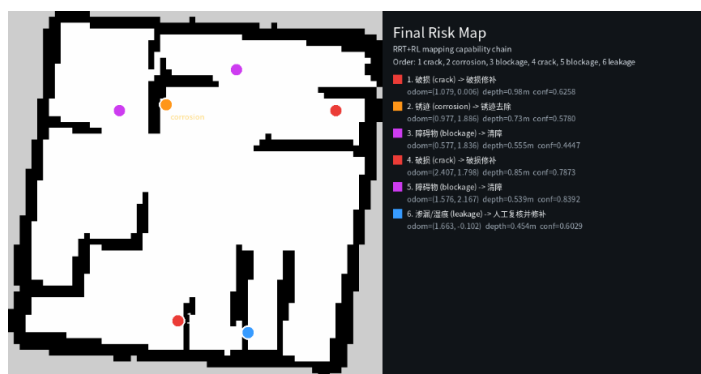


图 3-5 风险地图与风险点位置图

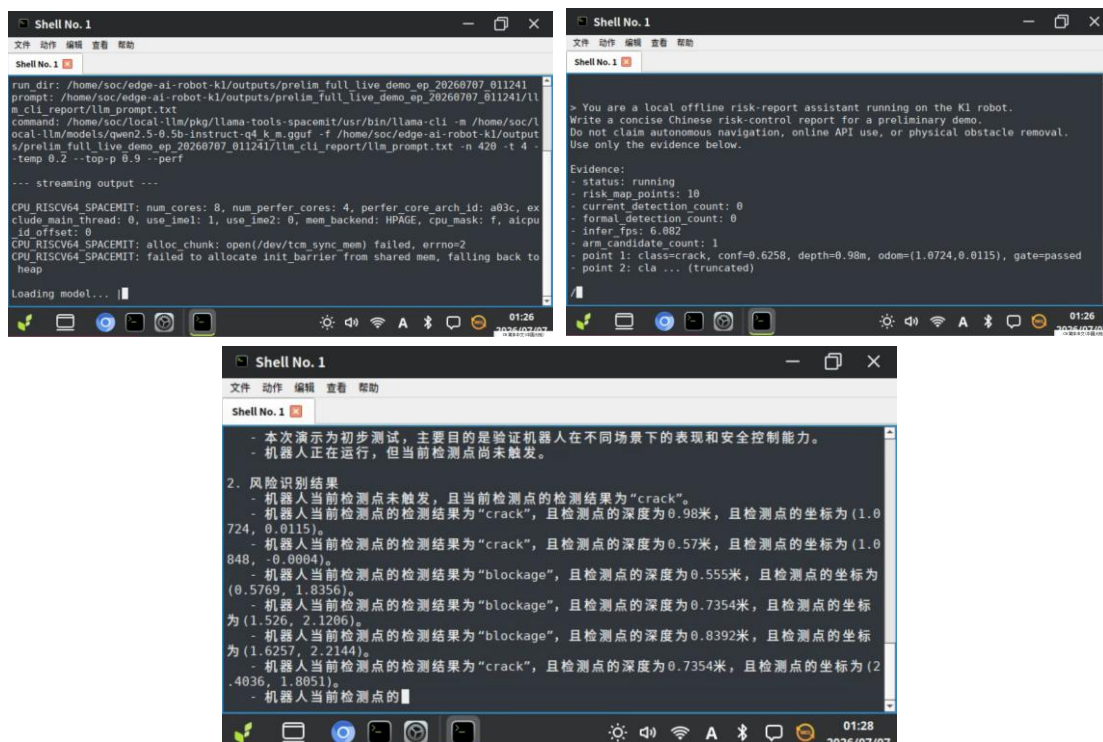


图 3-6 本地 LLM CLI 报告生成界面

3.3 特性成果

参见表 3-2，特性成果围绕建图精度、安全守护、本地推理、风险地图、机械臂响应和本地报告生成展开。系统不是单独展示一个模型或一个界面，而是把每个风险点从图像检测结果推进到地图坐标和人工处置建议。

表 3-2 系统功能与量化指标汇总

项目	实现结果	量化/记录指标
自主探测	SLAM-Frontier 探索逻辑	分辨率 0.05 m/pix；与实际一致
安全守护	经 /scan 距离过滤后发布 命令	小地图演示边界
本地视觉	D435 + YOLOv8n 量化 本地推理	mAP 0.949； EP 现场约 9-11 FPS
风险门限	confidence 与 depth 同时约束	
风险地图	bbox + depth + odom 投影地图	最终形成 6 个风险点
机械臂	MoveIt+RL 规划候选 + 安全	base_zero、人工确认、回安全位
本地报告	LLM CLI 根据风险点生成报告	风险类别、坐标、置信、总结

3.3.1 SLAM-Frontier 自主探测与安全守护成果

建图链路已经完成实机验证。系统通过 N10P 雷达和底盘里程计生成实时

/map，并使用 map_saver 保存为 pgm/yaml 文件。现场先单独验证地图，再接入风险识别，避免将地图误差和视觉误差混在一起判断。

SLAM-Frontier 自主探测与安全守护模块负责把未知环境探索、路径生成和底盘安全执行连接起来。

本地日志中可看到 scan_safety_guard_node 输出 “/input_vel -> /cmd_vel_guarded using /scan”以及 guard_status state=clear、front_min 等信息，证明底盘控制并非直接透传，而是经过前向距离约束后的受保护运动。

实验验证采用分阶段方式推进：T01 验证遥控建图启动，T02 验证雷达前向距离，T03 验证 odom 与地图变化，T04 保存并拉回地图，T05 接入 D435，T06 启动 YOLO 推理，T07 复核 confidence+depth gate，T08 生成风险事件，T09 输出风险地图与 LLM 报告。每一步都有日志、图像或 json 文件作为证据，避免只依赖现场口头描述。

3.3.2 D435 YOLO 本地推理成果

视觉模块使用 D435 获取 RGB 图像和深度图，在 K1 端加载 YOLOv8n 量化 ONNX 模型进行本地推理。模型通过 ONNX Runtime SpaceMITExecutionProvider 运行，不依赖云端 API，dashboard 中实时显示 infer_fps、latency 和检测框。

风险识别不仅依赖置信度，还同时检查目标深度范围。这样可以过滤过远、过近或深度不稳定的检测结果，避免风险点重复保存或错误落图。现场调试后，补光逻辑保持为人工按需开启，不再动态调整亮度，D435 自身曝光能够覆盖大部分亮暗变化。

端侧部署验证关注 infer_fps、latency、provider 日志和进程稳定性。SpaceMIT EP 可用时优先使用 EP 展示推理帧率，CPU provider 作为功能兜底；演示脚本启动前清理旧 RealSense、YOLO 和 HTTP 进程，避免 D435 或 EP 资源被多个进程同时占用。

3.3.3 风险空间化与风险地图成果

风险空间化模块将 YOLO bbox 中心点、深度值、相机参数和 odom 近似位姿转换为地图风险点。系统对同类近邻风险进行合并，避免同一处 crack、

corrosion、blockage 或 leakage 在连续帧中被重复记录。

最终风险图形成 R1 crack、R2 corrosion、R3 blockage、R4 crack、R5 blockage、R6 leakage 六类处置点。风险地图与 LLM 报告使用同一份结构化风险点，保证图上编号、类别和报告中的处置建议一致。

最终风险点按 R1-R6 组织，类别顺序为 crack、corrosion、blockage、crack、blockage、leakage。风险地图同时服务 UI、文档、LLM 和机械臂规划，避免出现图上一个点、报告另一个点、规划使用第三套坐标的问题。

3.3.4 机械臂与处置动作成果

机械臂部分已经形成风险类型到处置动作空间的映射：crack 对应破损修补和复查，corrosion 对应除锈和防腐处理，blockage 对应障碍物清理，leakage 对应查漏、封堵和干燥复查。当前实机展示对于 leakage 类型的风险采用 MoveIt+RL 规划候选响应，不在未验证接触力的情况下直接执行真实清障。

该设计使机械臂在系统中不是孤立动作，而是接收风险地图和风险类别后的执行候选。后续只需要进一步加入末端工具、接触力限制和真实清障安全验证，即可把当前响应扩展为真实处置。

MoveIt+RL 与动作原语采用分层关系：RL 或动作原语决定观察、预备、处置候选、回安全位等高层动作，具体轨迹由 MoveIt 在机械臂关节空间内求解，并接受工作空间、关节限位、碰撞边界、base_zero 和人工确认约束。RRT/A*/Nav2 保持在底盘建图/探索路径层。比赛阶段以 3D 打印物块响应展示处置闭环，不把未验证接触力的动作直接用于真实清障。

3.3.5 本地 LLM 报告与可视化成果

本地 LLM 报告模块从结构化风险点生成巡检结果，不进行开放式闲聊。报告逐项说明在地图近似坐标处发现的风险类别、置信度、风险含义和人工处置动作空间，最终形成“哪里需要清障、哪里需要修补、哪里需要除锈、哪里需要查漏”的人工任务清单。可视化成果包括 K1 本地 dashboard、Windows 端浏览器页面、风险位置图和 LLM CLI 录屏。展示顺序为先看 D435 YOLO overlay，再看风险点落图，最后看 LLM 按 token 流式生成报告，使评审能够直接看到本地推理和本地报告生成过程。

报告录制时将风险地图和 LLM CLI 放在同一段录屏中，等待完整输出或终端长时间无新增 **token** 后再停止。这样可以同时证明风险图已经生成、结构化风险点已经进入提示词、报告内容与地图编号一致，并且本地生成过程没有依赖云端 API。

第四部分 总结

4.1 综合总结

本作品面向地下管廊、仓储窄巷、设备间等 GPS 拒止、通信受限、弱照度及遮挡干扰场景，完成了一套以 K1 Muse Pi Pro 为核心的复杂受限空间离线认知机器人样机。系统以履带式移动底盘为载体，集成 N10P 单线激光雷达、Intel RealSense D435 RGB-D 相机、底盘里程计、三防补光模块和五轴总线舵机机械臂，形成了集移动、感知、计算、规划、执行和交互于一体的边缘智能平台。研发过程从低速遥控建图和基础传感器联调起步，逐步拓展至底盘安全运动、端侧风险识别、风险空间化、机械臂候选轨迹规划和本地报告生成，验证了系统在复杂环境中的集成能力与闭环运行能力。

1、移动与建图环节，系统融合 N10P 雷达扫描数据和底盘里程计信息，通过 slam_toolbox 完成二维环境地图构建，并在底盘速度链路中设置安全守护节点。上层遥控或 RRT/A*/Nav2 输出的速度指令必须经过障碍距离判断、速度裁剪和紧急停止逻辑后，才能发送至 C30D 底盘驱动板，从而降低狭窄空间中碰撞、误操作和通信异常带来的风险，从而进行路径搜索、自主导航和高层任务策略，使机器人依据地图结构、风险位置和安全状态执行巡检任务。

2、风险感知环节，D435 向 K1 提供 RGB 图像、深度图和相机内参，YOLOv8n 模型在 K1 本地完成裂缝、锈蚀、渗漏和障碍物识别。系统不只保存二维检测框，而是进一步融合目标深度、相机成像几何关系以及 odom/map 位姿，将视觉检测结果转换为带有风险类别、识别置信度、代表深度和地图坐标的结构化风险点。同时，系统利用置信度门限、depth gate 和同类近邻合并机制过滤无效目标，减少远距离误检、无效深度和连续视频流重复检测造成的冗余记录。由此，巡检结果由“检测到什么目标”提升为“风险位于哪里、是否需要处理以及后续如何处置”。

3、规划执行环节，风险地图点被转换为机械臂动作空间目标，由 MoveIt 结合 RRT/RRTConnect 等采样式规划方法生成接近观察、保持拍摄、处置预备、复核和返回安全位等候选轨迹。强化学习与动作原语训练仅作用于高层语义动作选择，不直接输出底盘速度、机械臂关节角度或舵机脉冲。真实执行仍需经过底盘

静止、关节软限位、目标可达性、工作空间校验、no-load 测试、串口写入确认和人工确认等安全门控。当规划失败、目标超出工作空间或通信状态异常时，系统进入 STOP_SAFE 状态，不强制执行机械臂动作。该设计既保留了策略学习和动作组合能力，也避免学习模块绕过底层安全链路，提高了动作过程的可解释性、可回退性和可复核性。

4、报告与人机交互环节，系统以结构化风险点为输入，通过 K1 本地 LLM 或确定性报告基线，生成包含风险坐标、风险类别、识别置信度、人工处置建议和机械臂动作空间的风险处置报告。处理结果可通过 K1 Dashboard、Windows 浏览器、风险地图、机械臂轨迹可视化和运行日志进行展示。识别、落图、规划、报告和可视化均可在本地完成，不依赖云端 API，使系统在弱网、断网或云端服务不可用时，仍能够保持基本巡检、风险记录和任务反馈能力。

作品形成了“环境建图—风险识别—空间定位—处置规划—报告生成—结果可视化”的完整端侧闭环。软件总体数据流图展示了传感器输入、ROS2 感知层、YOLO 风险识别、风险点空间化、机械臂规划和本地报告之间的数据关系；最终风险地图证明了二维视觉检测结果能够转换为可复核的地图坐标；YOLO overlay 展示了模型在实机图像上的识别能力；本地 LLM CLI 和 Dashboard 则展示了报告生成速度、风险信息 and 系统运行状态。上述结果共同说明，本作品并非多个功能模块的简单拼接，而是一套能够在车载端统一组织传感、推理、定位、规划、执行和人机交互的边缘智能机器人系统。

本项目的主要工程价值在于，将国产 RISC-V 边缘计算平台、多源传感器、轻量化视觉模型、二维风险地图、机械臂规划与本地大模型组织成可拆分、可调试、可恢复的系统架构。不同模块能够独立启动、独立验证和独立恢复，既便于实验阶段的问题定位，也提高了现场展示和连续运行过程中的稳定性。

4.2 可扩展之处

当前作品已经完成从环境感知到风险报告的样机闭环，但在定位精度、数据规模、自主能力和真实处置能力方面仍具有进一步提升空间。

1、可继续完善 D435、底盘和地图坐标系之间的 TF 标定，通过外参标定、时间同步和误差补偿，提高风险点由图像坐标向 odom/map 坐标转换的稳定性。

同时，可增加风险点多帧跟踪、概率融合和地图约束，降低单帧深度波动与底盘位姿误差对最终坐标的影响。

2、可扩充真实地下管廊、设备间和工业场景中的裂缝、锈蚀、渗漏与障碍物数据，增加不同照度、距离、材质、遮挡和拍摄角度下的样本，并通过模型量化、剪枝和算子适配，进一步降低推理延迟。后续也可将更多 YOLO 算子部署至 K1 平台的专用 AI 加速资源，更深入利用 SpaceMIT Execution Provider，提高 RISC-V 边缘端的推理效率和资源利用率。

3、在底盘运动方面，可在现有安全守护链路基础上，进一步接入更先进的自主定位、动态避障和任务级路径规划算法，使机器人发展为更强有力的自主巡检。

4、在机械臂方面，可将当前基于 3D 打印物块的 no-load 和轻型目标测试，推进到软物体、轻型障碍物和可控接触场景，并增加关节电流、舵机温度、末端接触、夹爪状态和力反馈检测。机械臂 RL 策略应继续限制在高层动作原语范围内，通过离线评估、安全回放和仿真到实机迁移验证后，再逐步扩大真实执行范围。

5、在交互与任务管理方面，可增加语音输入、任务调度、风险确认、人工复核和处置状态回写功能，使本地 LLM 不仅生成静态报告，还能够参与任务解释、巡检摘要和后续行动建议。未来还可探索多机器人共享地图、风险点协同复核和任务分配，使系统从单机巡检进一步发展为多机协同的复杂空间运维平台。相关技术路线可结合文献[1][2][7]进一步研究。

4.3 心得体会

本项目最大的工程难点，是如何将履带底盘、激光雷达、RGB-D 相机、AI 推理、二维地图、机械臂规划和本地 LLM 组织成稳定、可观察、可恢复的完整闭环。任一模块单独运行时相对容易，但进入实机联调后，会同时遇到传感器启动顺序、设备权限、USB 资源占用、串口稳定性、推理后端释放、地图尺度、深度有效性、坐标系转换和安全边界等综合问题。系统能否稳定工作，往往不只取决于模型精度，而取决于这些工程细节是否被清晰划分、持续记录并逐项验证。

在研发过程中，先冻结底盘安全链路，再逐步叠加认知与执行功能，是较为

稳妥的技术路线。项目首先验证 N10P 雷达、底盘里程计和 slam_toolbox 建图，再完成安全守护节点与速度控制链路；随后接入 D435 和 YOLOv8n，分别验证 RGB 图像、深度图、相机内参和模型推理结果；在视觉识别稳定后，再实现风险点空间化、风险地图、机械臂 MoveIt 规划和本地 LLM 报告。这样的分阶段集成方式能够将问题限制在较小范围内，避免多个模块同时接入后难以判断故障来源，也使每个阶段都能保留独立日志、结果文件和可视化证据。

K1 Muse Pi Pro 的使用，使我们对 RISC-V 边缘计算平台有了更具体的认识。最初容易将注意力集中在处理器核心数、RVV 向量扩展和 AI 算力等理论指标上，但实际部署过程中发现，真正决定系统体验的是操作系统、驱动、推理后端、算子支持、内存占用和外设稳定性之间的协同。K1 运行 Bianbu Linux 和 ROS2，能够在车载端同时承担传感器接入、YOLO 推理、风险点计算、日志管理、本地报告生成和 Dashboard 展示，但这也意味着必须对有限的计算、内存、USB 和显示资源进行统一调度。

在视觉模型部署方面，YOLOv8n 需要先导出为 ONNX，再通过 ONNX Runtime 接入 SpaceMIT Execution Provider。实际体验表明，EP 并不是简单地“打开加速”就能获得稳定性能，而需要检查模型算子是否被支持、计算图是否成功分配、是否存在 CPU 回退以及推理进程是否被正确释放。相机采集、EP 推理和浏览器可视化同时运行时，还可能出现 USB 占用、内存压力和旧进程残留等问题。因此，系统最终采用单 D435、单 EP 推理进程和独立运行目录的方式，并在每次启动前清理旧进程，以减少资源竞争。K1 端约 9~16 FPS 的 YOLO 推理结果，也让我们认识到，边缘部署不能只比较峰值 TOPS，更要观察完整数据链路中的实际帧率、延迟和连续运行稳定性。

本地 LLM 的部署同样体现了边缘计算的工程特点。K1 能够以约 5.8 tokens/s 的速度生成风险处置报告，证明系统可以在不依赖云端 API 的情况下完成结构化信息整理和文本生成。但当 LLM、YOLO、D435 和 Dashboard 同时运行时，系统资源压力会明显增大。因此，项目没有将所有任务强行并发执行，而是根据演示与任务流程进行阶段化调度，并保留确定性报告基线。这样的设计使本地 LLM 成为完整闭环中的报告生成模块，而不是为了展示模型而影响感知与控制链路的稳定性。

RRT、Nav2、MoveIt 和 RL 的分层引入，使系统从“识别风险”进一步走向“规划如何处置风险”。在底盘侧，RRT、A*或 Nav2 用于已知地图中的候选路径搜索、自主巡检和局部避障；在机械臂侧，MoveIt 及采样式规划器负责运动学、碰撞检测、关节限位和轨迹生成；RL 则负责高层动作原语选择，不直接控制底盘速度、关节角度或舵机脉冲。通过这种分层设计，风险地图中的每个风险点都能够与具体坐标、风险类别和动作空间建立联系，最终报告不再只是泛泛描述，而是能够说明哪里需要清障、哪里需要修补、哪里需要除锈以及哪里需要复核渗漏。

同时，我们还认识到，安全能力应当先于自主能力建立。对于履带底盘和机械臂组合系统，如果没有安全速度裁剪、base_zero、no-load、关节软限位、串口确认和人工确认等门控，即使上层规划结果正确，也可能因通信延迟、机械干涉、坐标误差或环境变化造成危险。因此，本作品将自主策略限制在安全状态机允许的范围内，使感知、规划学习模块都不能直接绕过底层约束。相比单纯追求更高层次的自动化，这种分层安全设计也为从仿真走向真实接触任务提供了清晰边界。

在展示策略上，风险位置图和本地 LLM CLI 是最直观的系统证据。YOLO overlay 回答“机器人看到了什么”，风险地图回答“风险在哪里”，底盘路径和 MoveIt 轨迹回答“机器人准备怎样到达和处置”，本地 LLM 报告回答“系统如何将结果整理为人工可读的任务清单”。当这些结果与 slam 地图、Dashboard 状态、推理帧率、token 速率和运行日志组合后，作品能够完整回答“看到了什么、位于哪里、准备如何处理、怎样形成报告”四个关键问题。

通过本项目，我们认识到，国产 RISC-V 边缘平台的价值并不只是替代，更是能够在受限功耗和本地运行条件下，将传感器数据、AI 推理、机器人规划和人机交互组织在同一可控设备上。同时也应看到，RISC-V 平台的软件生态、驱动兼容性、推理算子支持和工程工具链仍需要更多适配与验证。此次 K1 实机部署使我们从“模型能否运行”进一步关注“系统能否长期、稳定、可恢复地运行”，也使项目从单点算法验证转向软硬件协同设计。

边缘智能机器人真正的价值，在于把感知数据、空间位置、行动规划、安全约束和人机反馈连接起来。当识别结果能够被定位、被规划、被验证，并最终转化为可执行的处置建议时，端侧 AI 才真正从单点算法展示转变为具有实际任务价值的机器人系统。

从一块国产 RISC-V 开发板出发，我们完成了感知、推理、规划和执行的闭环，也亲身经历了国产平台从“能够运行”走向“稳定运行”所需要跨越的每一道工程门槛。今天的生态或许还不够成熟，但每一次部署、每一次调试、每一个真正运行起来的应用，都是它继续向前的一小步。

也许今天，我们还需要为一个驱动问题排查很久，为一个模型算子反复转换，为一次稳定运行仔细安排每个进程的启动顺序；但我们相信，正是无数开发者今天完成的这些看似琐碎的工作，会成为明天成熟生态的一部分。我们也希望，当未来的学生再次使用国产 RISC-V 平台搭建机器人时，他们面对的不再是“能不能运行”的疑问，而是能够更自由地思考“还能创造什么”。

也许今天的 K1 还不是终点，今天的系统也仍然只是一个样机。但正是这些并不完美的样机、并不完整的生态和仍需不断调试的开发过程，构成了技术真正向前迈进的道路。我们愿意相信，未来会有更强的芯片、更成熟的 EP 后端、更完善的驱动、更丰富的模型适配和更友好的开发环境。到那时，今天需要反复排查的问题，可能只需一次配置；今天需要谨慎取舍的资源，可能不再成为限制；今天在实验室中完成的端侧闭环，也许会真正走进管廊、工厂、仓库和灾害现场。

我们期待国产 RISC-V 不断突破算力、软件和生态边界，也期待未来的智能机器人能够拥有更多来自国产架构的“芯”。这份作品或许只是一次小规模尝试，但我们愿意相信，更多这样的尝试汇聚起来，就会成为国产 RISC-V 持续成长的力量。

我们期待国产 RISC-V 的成长，也愿意成为它成长过程中的一名普通使用者、一名实践者和一名记录者。或许我们的项目只是一次小规模尝试，但当一块国产芯片真正驱动机器人看见环境、理解风险并作出回应时，我们看到的不仅是一个作品的完成，更看到了一条仍在延伸的道路。我们相信，这条道路终将通向更加开放、自主、繁荣的国产计算生态。

这个项目带给我们的最后一份感受，是对国产 RISC-V 未来的一份更具体、更朴素的期待。我们期待它继续成长，也愿意在自己的能力范围内，成为这段成长过程中的使用者、记录者和参与者。因为一项技术真正拥有未来，不只是因为有人提出了它，更是因为有人愿意相信它、使用它、完善它，并把它带到一个又一个真实的场景之中。

第五部分 参考文献

- [1] 进迭时空. Bianbu 操作系统文档[EB/OL]. (2026-07-09)[2026-07-09].
<https://bianbu.spacemit.com/>.
- [2] 进迭时空. SpaceMIT 官方网站[EB/OL]. (2026-07-09)[2026-07-09].
<https://www.spacemit.com/>.
- [3] Open Robotics. ROS 2 Humble documentation[EB/OL]. (2026-07-09)[2026-07-09].
<https://docs.ros.org/en/humble/>.
- [4] Steve Macenski, et al. slam_toolbox[EB/OL]. (2026-07-09)[2026-07-09].
https://github.com/SteveMacenski/slam_toolbox.
- [5] Navigation2 Project. Navigation2 documentation[EB/OL]. (2026-07-09)[2026-07-09]. <https://navigation.ros.org/>.
- [6] Intel RealSense. ROS Wrapper for RealSense Cameras[EB/OL]. (2026-07-09)[2026-07-09]. <https://github.com/realsenseai/realsense-ros>.
- [7] Microsoft. ONNX Runtime documentation[EB/OL]. (2026-07-09)[2026-07-09].
<https://onnxruntime.ai/docs/>.
- [8] Ultralytics. Ultralytics YOLO documentation[EB/OL]. (2026-07-09)[2026-07-09].
<https://docs.ultralytics.com/>.
- [9] MoveIt Project. MoveIt 2 Documentation[EB/OL]. (2026-07-09)[2026-07-09].
<https://moveit.picknik.ai/main/index.html>.
- [10] ggml-org. llama.cpp: LLM inference in C/C++[EB/OL]. (2026-07-09)[2026-07-09]. <https://github.com/ggml-org/llama.cpp>.
- [11] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. 2nd ed. Cambridge: MIT Press, 2018.